Newtonsoft.Json Array Filtering with JSONPath Tutorial

Overview

JSONPath is a query language for JSON, similar to XPath for XML. Newtonsoft.Json (Json.NET) provides powerful JSONPath support for filtering and selecting data from JSON arrays and objects.

Installation

```bash
Install-Package Newtonsoft.Json
```

Basic Syntax

JSONPath expressions start with `$` (representing the root) and use dot notation or bracket notation to navigate through the JSON structure.

Common Operators

Operator	Description	Example
`$`	Root element	`$`
`.`	Child element	`$.store.book`
`[]`	Array index or filter	`$[0]` or `$[?(@.price < 10)]`
`*`	Wildcard	`$.store.*`
`..`	Recursive descent	`$..price`
`?()`	Filter expression	`$[?(@.price < 10)]`
`@`	Current element in filter	`@.price`

Sample Data

Let's use this JSON data for our examples:

```json
```

```
{
  "store": {
    "books": [
      {
        "id": 1,
        "title": "The Great Gatsby",
        "author": "F. Scott Fitzgerald",
        "price": 12.99,
        "category": "fiction",
        "inStock": true,
        "tags": ["classic", "american"]
      },
      {
        "id": 2,
        "title": "To Kill a Mockingbird",
        "author": "Harper Lee",
        "price": 14.99,
        "category": "fiction",
        "inStock": false,
        "tags": ["classic", "drama"]
      },
      {
        "id": 3,
        "title": "1984",
        "author": "George Orwell",
        "price": 13.99,
        "category": "dystopian",
        "inStock": true,
        "tags": ["dystopian", "political"]
      },
      {
        "id": 4,
        "title": "The Catcher in the Rye",
        "author": "J.D. Salinger",
        "price": 11.99,
        "category": "fiction",
        "inStock": true,
        "tags": ["coming-of-age"]
      }
    ],
    "electronics": [
      {
        "id": 101,
        "name": "Laptop",
        "brand": "TechCorp",
        "price": 999.99,
```

```
      "inStock": true
    },
    {
      "id": 102,
      "name": "Smartphone",
      "brand": "PhoneCorp",
      "price": 699.99,
      "inStock": false
    }
  ]
 }
}
```

Basic Array Filtering Examples

1. Select All Books

```csharp
using Newtonsoft.Json.Linq;

string json = /* your JSON data */;
JObject data = JObject.Parse(json);

// Get all books
var allBooks = data.SelectTokens("$.store.books[*]");
foreach (var book in allBooks)
{
    Console.WriteLine(book["title"]);
}
```

2. Select Books by Index

```csharp
// First book
var firstBook = data.SelectToken("$.store.books[0]");

// Last book
var lastBook = data.SelectToken("$.store.books[-1]");

// First two books
var firstTwoBooks = data.SelectTokens("$.store.books[0,1]");

// Books from index 1 to 2
var rangeBooks = data.SelectTokens("$.store.books[1:3]");
```

Filter Expressions

3. Price-Based Filtering

```csharp
// Books cheaper than $13
var cheapBooks = data.SelectTokens("$.store.books[?(@.price < 13)]");

// Books more expensive than $14
var expensiveBooks = data.SelectTokens("$.store.books[?(@.price > 14)]");

// Books between $12 and $14
var midPriceBooks = data.SelectTokens("$.store.books[?(@.price >= 12 && @.price <= 14)]");
```

4. String-Based Filtering

```csharp
// Books by specific author
var orwellBooks = data.SelectTokens("$.store.books[?(@.author == 'George Orwell')]");

// Fiction books
var fictionBooks = data.SelectTokens("$.store.books[?(@.category == 'fiction')]");

// Books with titles containing specific words
var gatsbyBooks = data.SelectTokens("$.store.books[?(@.title =~ /.*Gatsby.*/i)]");
```

5. Boolean and Null Filtering

```csharp
// Books in stock
var inStockBooks = data.SelectTokens("$.store.books[?(@.inStock == true)]");

// Books not in stock
var outOfStockBooks = data.SelectTokens("$.store.books[?(@.inStock == false)]");

// Books with specific property existing
var booksWithTags = data.SelectTokens("$.store.books[?(@.tags)]");
```

Advanced Filtering

6. Array Property Filtering

```csharp
```

```csharp
// Books with specific tags
var classicBooks = data.SelectTokens("$.store.books[?(@.tags[*] == 'classic')]");

// Books containing "classic" tag
var classicBooksAlt = data.SelectTokens("$.store.books[?('classic' in @.tags)]");

// Books with more than one tag
var multiTagBooks = data.SelectTokens("$.store.books[?(@.tags.length > 1)]");
```

7. Complex Logical Operations

```csharp
// Fiction books under $15 that are in stock
var availableFiction = data.SelectTokens("$.store.books[?(@.category == 'fiction' && @.price < 15 && @.inStock =

// Books by multiple authors
var specificAuthors = data.SelectTokens("$.store.books[?(@.author == 'George Orwell' || @.author == 'Harper Lee

// Books NOT in fiction category
var nonFiction = data.SelectTokens("$.store.books[?(@.category != 'fiction')]");
```

8. Recursive Search

```csharp
// All prices in the entire document
var allPrices = data.SelectTokens("$..price");

// All items in stock (books and electronics)
var allInStock = data.SelectTokens("$..[?(@.inStock == true)]");

// All items with names or titles
var allNamed = data.SelectTokens("$..[?(@.name || @.title)]");
```

Practical Usage Examples

Complete C# Example

```csharp
```

```
using System;
using System.Linq;
using Newtonsoft.Json.Linq;

class Program
{
    static void Main()
    {
        string json = /* your JSON data */;
        JObject data = JObject.Parse(json);

        // Example 1: Get titles of fiction books under $15
        var affordableFiction = data.SelectTokens("$.store.books[?(@.category == 'fiction' && @.price < 15)]")
                                    .Select(book => book["title"].ToString())
                                    .ToList();

        Console.WriteLine("Affordable Fiction Books:");
        affordableFiction.ForEach(title => Console.WriteLine($"- {title}"));

        // Example 2: Calculate average price of in-stock items
        var inStockPrices = data.SelectTokens("$..[?(@.inStock == true)].price")
                                .Select(price => (decimal)price)
                                .ToList();

        if (inStockPrices.Any())
        {
            decimal avgPrice = inStockPrices.Average();
            Console.WriteLine($"\nAverage price of in-stock items: ${avgPrice:F2}");
        }

        // Example 3: Group books by category
        var booksByCategory = data.SelectTokens("$.store.books[*]")
                                  .GroupBy(book => book["category"].ToString())
                                  .ToDictionary(g => g.Key, g => g.Count());

        Console.WriteLine("\nBooks by category:");
        foreach (var category in booksByCategory)
        {
            Console.WriteLine($"- {category.Key}: {category.Value} books");
        }
    }
}
```

Performance Tips

1. Use Specific Paths

```csharp
// Better: Specific path
var books = data.SelectTokens("$.store.books[?(@.price < 15)]");

// Avoid: Recursive search when not needed
var books = data.SelectTokens("$..[?(@.price < 15)]");
```

2. Cache Compiled Expressions

```csharp
// For repeated queries, consider caching the path compilation
// This is handled internally by Json.NET, but be aware of performance implications
```

3. Limit Results Early

```csharp
// Use Take() to limit results if you only need a few
var firstFiveCheapBooks = data.SelectTokens("$.store.books[?(@.price < 20)]")
                              .Take(5);
```

Common Filter Operators

Operator	Description	Example
`==`	Equals	`@.price == 12.99`
`!=`	Not equals	`@.category != 'fiction'`
`<`	Less than	`@.price < 15`
`<=`	Less than or equal	`@.price <= 15`
`>`	Greater than	`@.price > 10`
`>=`	Greater than or equal	`@.price >= 10`
`=~`	Regex match	`@.title =~ /.*Gatsby.*/i`
`in`	Contains (for arrays)	`'classic' in @.tags`
`&&`	Logical AND	`@.price < 15 && @.inStock`
`||`	Logical OR	`@.category == 'fiction' || @.category == 'drama'`

Error Handling

```csharp
```

```
try
{
    var results = data.SelectTokens("$.store.books[?(@.price < invalidValue)]");
}
catch (JsonException ex)
{
    Console.WriteLine($"JSONPath error: {ex.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"General error: {ex.Message}");
}
```

Best Practices

1. **Start Simple**: Begin with basic path expressions and add complexity gradually
2. **Test Expressions**: Use online JSONPath testers to validate your expressions
3. **Handle Nulls**: Always check for null results when accessing properties
4. **Use Strongly Typed Models**: Consider deserializing to strongly typed objects when possible
5. **Cache Results**: Store frequently used query results to improve performance
6. **Validate Input**: Ensure your JSON structure matches your JSONPath expectations

This tutorial covers the most common scenarios for filtering JSON arrays with Newtonsoft.Json and JSONPath expressions. The key is to understand the filter syntax and combine operators effectively to create precise queries for your data.